Temper

Compile-Time Security for Enterprise Infrastructure

One source of truth. Every language your organization ships. Zero security drift.

JavaScript | Python | Rust | Java | Lua | C# | and more to come...

THE PROBLEM

Multi-language SDKs are an expanding attack surface

What enterprises face today

- Security-critical logic duplicated across **5–12 language SDKs**
- Each SDK is an independent codebase with independent vulnerabilities
- A single CVE requires coordinated patches across every language
- Vendor SDK teams release on different cadences — drift is inevitable

The cost of drift

- Serialization bug in one SDK goes unpatched for **months** in another
- Auth token validation differs subtly between languages
- Compliance teams must audit **N codebases** instead of one
- Major version migrations span **years**, leaving old versions unpatched

Every language-specific implementation is a new place for a vulnerability to hide. The industry treats this as unavoidable. **It is not.**

SUPPLY CHAIN RISK

Your dependencies are only as secure as their weakest SDK

84%

of codebases contain at least one known vulnerability

N×

multiplier for every language you ship

When a vendor ships SDKs in 10 languages, a vulnerability in their serialization layer exists in 10 independent implementations. Patching one does not patch the others. Your supply chain risk **multiplies with every language target**.

Temper collapses that multiplier to **1**.

Source: Synopsys OSSRA 2024

THE ANSWER

One source. Six backends. Zero drift.

Temper compiles a single source tree to idiomatic, typed, importable libraries in 6 languages. Not transpilation — native output.

Temper Source → `temper build` → ES Modules | Python 3 | Cargo Crate | Java | Lua | .NET

Traditional multi-SDK

- N codebases × N security reviews
- Per-language runtime teams
- Coordination overhead per CVE
- Semantic drift between implementations

Temper

- **1 codebase = 1 security surface**
- One team, one review, one audit
- Fix once, rebuild, ship everywhere
- Behavioral equivalence by construction

PROOF

A type-safe SQL ORM, compiled to 6 backends

We built a security-focused ORM with 5 defense layers, compiled it to 6 languages, and deployed 6 identical applications. Each underwent independent MITRE ATT&CK / CWE analysis.

LANGUAGE	FRAMEWORK	SQL INJECTION VULNS	CWE FINDINGS
JavaScript	Express + EJS	**0**	**0 critical**
Python	Flask + Jinja2	**0**	**0 critical**
Rust	Axum + askama	**0**	**0 critical**
Java	Spring Boot	**0**	**0 critical**
Lua	Raw socket HTTP	**0**	**0 critical**
C#	ASP.NET Razor	**0**	**0 critical**

Same security properties. Same behavior. **Six independent audits confirm zero drift.**

SECURITY ARCHITECTURE

5 compile-time defense layers

	LAYER	CWE	WHAT IT PREVENTS
1	**SafeIdentifier**	CWE-89	Identifier injection. Regex-validated sealed type — never raw strings.
2	**Typed SqlPart**	CWE-89	Value injection. Sealed interface with exhaustive type dispatch.
3	**SqlBuilder**	CWE-89	Structural injection. No raw append. Structure and data strictly separated.
4	**Changeset**	CWE-915	Mass assignment. Whitelist-only field access via typed identifiers.
5	**Query Builder**	CWE-400	Unbounded queries. WHERE clause accepts only typed fragments.

These defenses are **structural, not advisory**. The type system makes unsafe construction impossible — in every compiled backend, simultaneously.

INCIDENT RESPONSE

1 vulnerability. 1 commit. 6 backends patched.

MITRE analysis found 3 ORM-level concerns. All 3 were fixed in a single Temper commit and rebuilt across all backends.

6

PRs, reviews, releases

Traditional multi-SDK

VS

1

commit, build, done

Temper

```
// One change in Temper source ~~colNames.add(pair.key); // raw String — unvalidated~~ colNames.add(fd.name.sqlValue);
// SafeIdentifier → validated // Propagated to JS, Python, Rust, Java, Lua, C# in a single build
```

Mean time to remediation drops from **weeks of coordination** to **minutes of compilation**.

AI writes code. Temper makes it safe.

AI-generated code follows the path of least resistance. In most languages, that path leads directly to injection vulnerabilities.

9

Critical/High vulns

AI-generated ORM (Ruby, 3–4 prompts)

VS

0

Injection vulns

AI-generated ORM (Temper, comparable effort)

Same AI. Same goals. Same effort. The difference is that Temper's type system **makes the unsafe path inexpressible**. AI cannot accidentally skip validation when the compiler enforces it.

> *Enterprise security cannot depend on every developer — or every AI — remembering to do the right thing.*

COMPLIANCE & GOVERNANCE

One audit surface. Every language.

Traditional audit burden

- Separate SAST/DAST per language SDK
- Independent pen tests for each implementation
- Per-language dependency scanning (npm, PyPI, Maven, NuGet...)
- Compliance evidence multiplied by target count

Temper audit surface

- **One source** to review, scan, and certify
- Compiler is the single point of trust
- Behavioral equivalence proven by construction
- SOC 2, FedRAMP, PCI evidence generated once

When regulators ask “how do you ensure consistent security across platforms?” the answer is not process. **The answer is the compiler.**

ENTERPRISE APPLICATIONS

Where Temper eliminates risk

SDK & API Libraries

Auth flows, serialization, retry logic, rate limiting — write once, ship to every language your customers use. One CVE response, not twelve.

Security Primitives

Token validation, input sanitization, crypto wrappers — the code that *must* be correct everywhere gets one implementation and one audit.

Data Validation & Policy

Schema validators, format parsers, business rule engines — consistent enforcement across frontend, backend, mobile, and embedded.

Regulated Infrastructure

Financial transaction logic, healthcare data handling, identity verification — compile-time guarantees for code that regulators scrutinize.

RETURN ON INVESTMENT

Security engineering that scales down, not up

METRIC	TRADITIONAL (N SDKS)	TEMPER (1 SOURCE)
Security reviews per release	N	**1**
CVE response time	Weeks (coordinated)	**Minutes (rebuild)**
Compliance audit scope	N codebases	**1 codebase + compiler**
Behavioral drift between languages	Inevitable	**Impossible by construction**
Engineers needed per language	Dedicated team	**0 — compiler handles it**
Major version migration	Years	**One build cycle**

The cost of multi-language security scales linearly with traditional tooling. With Temper, **it is constant**.

The numbers

0
injection vulnerabilities
across 6 backends

1
commit to fix all findings
in every language

5
compile-time
defense layers

9
critical vulns in conventional
AI-generated code

6
language targets
from single source

1
audit surface
for compliance

Enterprise security is not a process problem. It is an architecture problem.

Temper makes security a compile-time guarantee, not a runtime hope.

github.com/temperlang/temper